

BOSS HUNTER MMO RPG

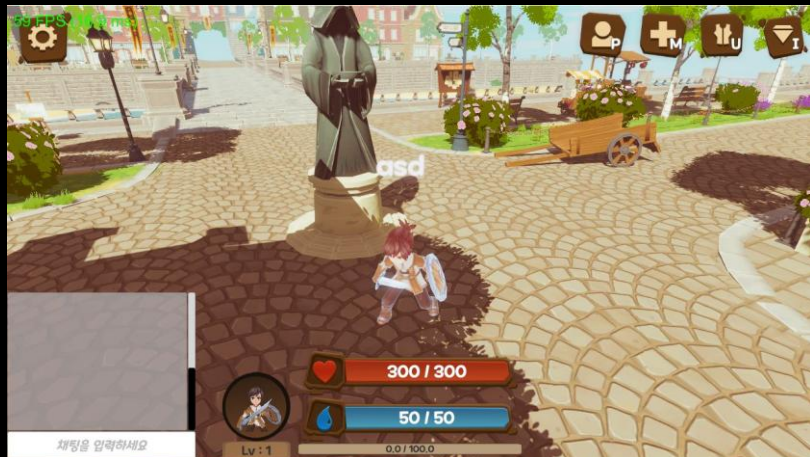
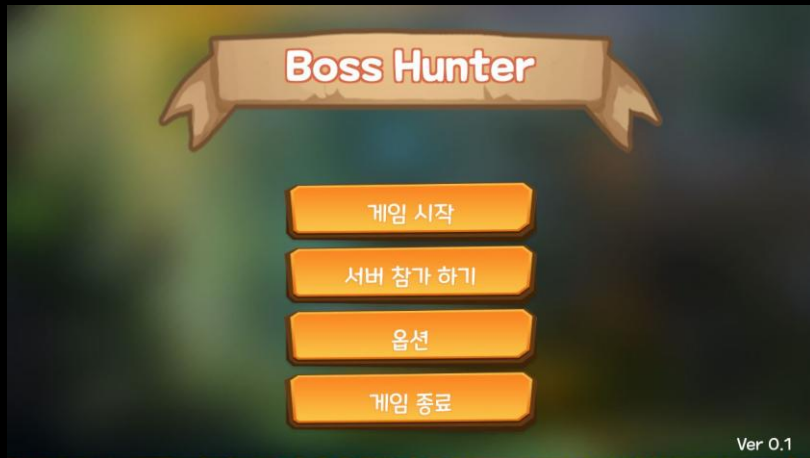
작성자 : 안승현

목차

1. BossHunter 포트폴리오 소개
2. Mirror API
3. 싱글톤 매니저 클래스
4. 컨텐츠 클래스 설계
5. 아이템
6. 플레이어와 몬스터

7. 보스 몬스터 & 스킬 구현
8. 플레이어 물리 움직임
9. 매달리기 & 벽 점프
10. 씬끼리 유저 통신
11. 인스턴스 씬
12. 유튜브, 깃허브 링크

1. BOSSHUNTER 포트폴리오 소개



- 사용 엔진 : UNITY
- 엔진 버전 : 2021.3.18f1 LTS
- 사용 언어 : C#
- 작업 인원 : 1명
- 작업 영역 : 콘텐츠 제작, 디자인, 기획
- 장르 : MMORPG
- 소개 : 미래 API + 유니티 멀티플레이 MMORPG
- 플랫폼 : PC
- 형상관리 : GitHub



2. MIRROR API

- Unet으로부터 포크(fork)해 다수의 개발자들이 오픈 소스로 제작한 Mirror
 - 무료 오픈 소스 게임 네트워크 라이브러리
 - 오픈 소스 API라 누구나 사용법만 알면 네트워크 게임을 제작할 수 있는 에셋
- ✓ Mirror API를 효율적으로 사용하기 위해
Mirror API 문서, 공식 Discord, 영상 매체를 통해 개념을 학습하여
해당 포트폴리오에 적용시켰습니다.

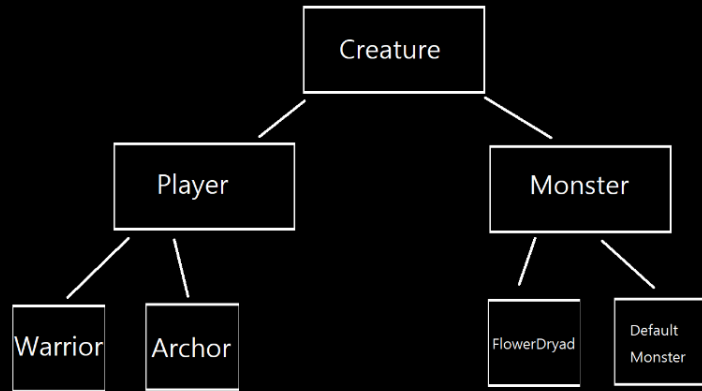
3. 싱글톤 매니저 클래스

```
public class Managers : NetworkManager
{
    static Managers instance;
    참조 62개
    public static Managers Instance
    {
        get
        {
            Init();
            return instance;
        }
    }
}
```

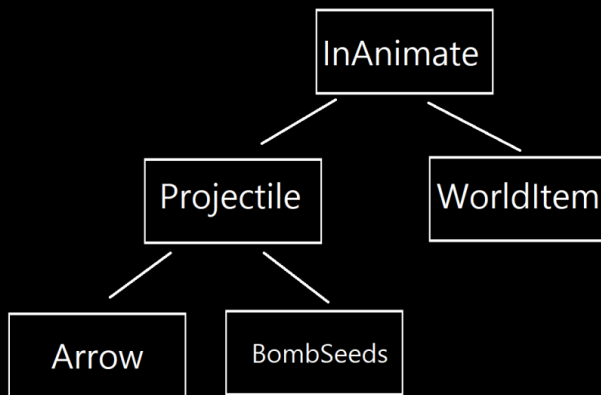
```
DataManager _data = new DataManager();
InputManager _input = new InputManager();
PoolManager _pool = new PoolManager();
ResourceManager _resource = new ResourceManager();
SceneManagerEx _scene = new SceneManagerEx();
SoundManager _sound = new SoundManager();
UIManager _ui = new UIManager();
```

- 유니티 엔진은 언리얼 엔진에 비해 게임을 위한 기능들이 구현되어 있지 않다. (ex)오브젝트 풀링)
- 모든 전역 매니저 클래스를 관리 및 접근이 용이한 싱글톤 매니저를 생성 및 정의한다
- 전역 클래스는 주로 유일성을 보장 및 특정 목적에 맞게 작동하는 기능들을 구현 후 접근하기 쉽게 싱글톤 매니저에서 전역 변수로 정의 및 초기화

소스코드 : [Managers.cs](#)



소스코드 : [Creature 폴더](#)



소스코드 : [InAnimate 폴더](#)

4. 콘텐츠 클래스 설계

- 콘텐츠에 사용될 클래스 정의
 - 생명체 클래스 (플레이어, 몬스터)
 - 비생명체 클래스 (투사체, 월드 아이템)
- 중복 코드 방지 및 생산성을 위해 상속 관계로 생성 및 설계
- 각 객체가 초기화 될 때 현재 씬의 물리 시스템을 넣어준다. (서버에서 서로 다른 씬의 물리 처리를 막기 위함)

5. 아이템



- RPG 핵심 요소인 플레이어가 사용할 아이템 구현
- 아이템 데이터 클래스와 아이템 데이터를 가지고 기능을 구현하는 클래스를 대칭적으로 정의 및 구현
- 아이템 데이터는 CSV으로 정의 및 내보내기 후 에디터 특정 위치에 파일을 배치하면 런타임에 데이터를 읽어 메모리에 적재

소스 코드 : [아이템 폴더](#)

소스 코드 : [아이템 데이터 폴더](#) 144줄~

소스 코드 : [아이템 데이터 CSV](#)

6. 플레이어와 몬스터 구현

참조 4개

```
public void DefaultState(Define.State defaultState, BaseState baseState)...
```

참조 29개

```
public void RegisterState(Define.State state, BaseState baseState)...
```

참조 3개

```
public void ChangeDefaultState()...
```

참조 62개

```
public bool ChangeState(Define.State _nextState, bool _checkCondition = true)...
```

참조 0개

```
public void RemoveState(Define.State removeState)...
```

소스코드 : [State Machine](#)

참조 5개

```
public BaseState(Define.State _state, Creature _creature, BaseController _controller)...
```

참조 14개

```
public virtual bool CheckCondition()...
```

참조 20개

```
public abstract void EnterState();
```

참조 19개

```
public abstract void FixedUpdateState();
```

참조 19개

```
public abstract void UpdateState();
```

참조 19개

```
public abstract void ExitState(Define.State _state, BaseState baseState);
```

소스코드 : [Base State](#)

- 플레이어와 몬스터를 제어하기 위해 FSM(유한 상태 머신) 사용
한번에 한번의 상태만 가질 수 있는 특징이 있다.
- 상태를 정의하는 BaseState 추상 클래스
상태를 제어하는 StateMachine 추상 클래스 구현
- 구현 및 등록
 - 구현하고자 하는 기능이 있을시 BaseState를 상속받아 추상 함수를 구현하면 된다.
 - 구현된 기능은 StateMachine에 등록하여 사용

7. 보스 몬스터 & 스킬 구현



- RPG게임에 보스는 빠질 수 없고 랜덤한 패턴의 몬스터를 구현하기 위함
- 기존 몬스터의 FSM와 동일한 로직에 아래와 같은 기능 추가
 - 새로운 스킬 상태 추가
 - 사용가능한 스킬 리스트 반환 로직
 - 스킬 리스트 중에서 랜덤으로 스킬을 선택하는 로직

```

[ServerCallback]
protected override void InitStateMachine()
{
    MonsterIdleState m_idleState = new MonsterIdleState(Define.State.Idle, this, controller);
    stateMachine.DefaultState(Define.State.Idle, m_idleState);

    MonsterChaseState m_chaseState = new MonsterChaseState(Define.State.Chase, this, controller);
    stateMachine.RegisterState(Define.State.Chase, m_chaseState);

    MonsterReturnState m_returnState = new MonsterReturnState(Define.State.Return, this, controller);
    stateMachine.RegisterState(Define.State.Return, m_returnState);

    MonsterPatrolState m_patrolState = new MonsterPatrolState(Define.State.Patrol, this, controller);
    stateMachine.RegisterState(Define.State.Patrol, m_patrolState);

    MonsterAttackState m_attackState = new MonsterAttackState(Define.State.NormalAttack, this, controller);
    stateMachine.RegisterState(Define.State.NormalAttack, m_attackState);

    // skill
    MonsterDissemination m_dissemination = new MonsterDissemination(Define.State.Skill, this, controller, 50,
    stateMachine.RegisterState(Define.State.Skill, m_dissemination);

    HitState m_hitState = new HitState(Define.State.Hit, this, controller);
    stateMachine.RegisterState(Define.State.Hit, m_hitState);

    DeadState m_deadState = new DeadState(Define.State.Dead, this, controller);
    stateMachine.RegisterState(Define.State.Dead, m_deadState);
}

```

소스코드 : [FlowerDrad.cs](#)

- Monster 상속한 FlowerDryad 보스 클래스 생성
- InitStateMachine 재정의
 - Monster 클래스와 다르게 SkillState 추가
 - MonsterDissemination (스킬) -

전방에 닿거나 일정 시간이 지나면 터지는
씨 폭탄을 뿌립니다.

```

public class MonsterIdleState : BaseStateMonster
{
    // 패트롤 대기 시간
    float patrolWaitTime;

    // 대상 스킬 ENUM
    Define.State skillEstate;

    참조 2개
    public MonsterIdleState(Define.State _state, Monster _monster, MonsterController _monsterController) ...

    참조 4개
    public override void EnterState()
    {
        MonsterGS.GetNetAnim().animator.SetFloat(Managers.AnimHash.FmoveSpeed, 0.0f);

        // 패트롤 대기 시간을 랜덤으로 설정한다.
        patrolWaitTime = Random.Range(3.0f, 10.0f);

        // 아이들 상태 진입시 사용할 수 있는 스킬 하나 정하기
        skillEstate = MonsterGS.MonsterStateMachine.RandomSelectSkill(MonsterGS.StateMachine.UseableSkill());
    }
}

```

소스코드 : [MonsterIdleState.cs](#)

```

[ServerCallback]
참조 2개
public Define.State RandomSelectSkill(List<Define.State> list)
{
    if (list == null || list.Count == 0)
        return Define.State.None;

    int index = Random.Range(0, list.Count + 1);

    if (index >= list.Count)
        return Define.State.None;

    return list[index];
}

```

소스코드 : [MonsterStateMachine.cs](#)

• 스킬 사용 순서

• Idle 상태 진입시 (EnterState)

- UseableSkill()를 통해 StateMachine에 등록된 스킬 리스트 반환
- RandomSelectSkill()을 통해 스킬 리스트와 + 스킬을 사용하지 않는 경우의 수를 포함해 랜덤으로 상태를 반환한다.

• Idle 상태 진행중 (FixedUpdateState)

- 고정 프레임 호출 함수
 - 대상 존재시
 - 스킬 사용 시도 성공하면 상태 전환
 - 공격 사거리 체크
 - 사정거리면 공격 시도 후 성공하면 상태 전환
 - 비사정거리면 추적 상태 전환
 - 대상이 없을시
 - 일정 시간 대기 후 패트롤

8. 플레이어 물리 움직임



- 역동적인 움직임을 구현하기 위해 Rigidbody 기반 움직임 채택
- 구현 핵심 로직
 - 플레이어 경사 이동 제한
 - 경사 이동시 방향벡터 프로젝트션
 - 언덕 미끄럼 제어

플레이어 경사 이동 제한

```
protected Vector3 GetDirection(float currentMoveSpeed)
{
    IsOnSlope = IsSlopeCheck();
    IsOnGround = IsGroundCheck();

    // 다음 프레임 위치 미리 계산하여 언덕인지 확인하여 이동할지 말지 결정
    Vector3 calculatedDirection =
        CalculateNextFrameGroundAngle(currentMoveSpeed) < maxSlopeAngle ?
        inputDirection : Vector3.zero;

    // 다음 프레임 위치에서 구한 방향이 현재 경사진곳이고 땅이면 프로젝션 방향을 구한다.
    calculatedDirection = (IsOnGround && IsOnSlope) ?
        AdjustDirectionToSlope(calculatedDirection) : calculatedDirection;

    return calculatedDirection;
}
```

참조 1개

```
public float CalculateNextFrameGroundAngle(float moveSpeed)
{
    // 다음 프레임 캐릭터 앞 부분 위치
    var nextFramePlayerPosition = precedingSlotCheck.position + inputDirection
        * moveSpeed * Time.fixedDeltaTime;

    // 다음 프레임 위치에서 경사인지 체크한다.
    if(creature.GetSetPhysics.Raycast(nextFramePlayerPosition, Vector3.down, out RaycastHit hitInfo,
        PRECEDING_RAY_DISTANCE, Managers.LayerManager.Ground))
    {
        return Vector3.Angle(Vector3.up, hitInfo.normal);
    }

    return 0f;
}
```

- 벽이나 급경사에 올라가지 못하게 제한을 두기 위해 구현
- 입력이 들어오면 다음 프레임 이동 위치에서 아래 방향으로 Ray를 발사하여 충돌체의 Normal 벡터를 구한다.
- Normal 벡터와 윗 방향 벡터의 내적을 구해 언덕의 각도를 구한다.
- 플레이어가 올라갈 수 있는 언덕 각도 보다 크면 해당 방향 벡터를 0으로 초기화 하여 움직일 수 없게 한다.

소스코드 : [PlayerController.cs](#)



빨간 점 : Ray 시작 위치

빨간 선 : Ray를 `Vector3.Down` 방향으로 발사한 방향

초록 선 : 피격위치의 Normal 벡터

파란 선 : `Vector3.up`

파란선과 초록선을 내적 + `ACOS`로 각도를 구해 움직임

여부 판단

경사 이동시 방향벡터 프로젝션

```
protected Vector3 GetDirection(float currentMoveSpeed)
{
    IsOnSlope = IsSlopeCheck();
    IsOnGround = IsGroundCheck();

    // 다음 프레임 위치 미리 계산하여 언덕인지 확인하여 이동할지 말지 결정
    Vector3 calculatedDirection =
        CalculateNextFrameGroundAngle(currentMoveSpeed) < maxSlopeAngle ?
        inputDirection : Vector3.zero;

    // 다음 프레임 위치에서 구한 방향이 현재 경사진곳이고 땅이면 프로젝션 방향을 구한다.
    calculatedDirection = (IsOnGround && IsOnSlope) ?
        AdjustDirectionToSlope(calculatedDirection) : calculatedDirection;

    return calculatedDirection;
}
```

```
// 해당 방향을 현재 땅의 노말벡터로 프로젝션 한다.
참조 1개
public Vector3 AdjustDirectionToSlope(Vector3 direction)
{
    return Vector3.ProjectOnPlane(direction, slopeHit.normal);
}
```

- 플레이어가 언덕을 오를때 튀어오르는 문제를 해결하기 위해 구현
- 플레이어가 땅에 있고 경사진곳 이면
 - 해당 경사진 곳의 Normal 벡터 기준으로 방향벡터를 프로젝션(좌표이동)한다.
- 플레이어가 땅에 있지 않고 또는 경사진곳이 아니면
 - 입력한 방향 벡터로 이동

소스코드 : [PlayerController.cs](#)



빨간점 : Ray 시작 위치

빨간선 : Ray를 `Vector3.down`으로 발사한 방향

초록선 : 피격위치의 Normal 벡터

노란선 : 입력 방향

갈색선 : Normal 벡터의 기준으로 프로젝션한 방향

언덕 미끄럼 제어

```
// 땅인지 피직스 체크박스로 확인
참조 1개
public bool IsGroundCheck()
{
    Vector3 boxSize = new Vector3(
        transform.lossyScale.x * 0.2f,
        0.01f,
        transform.lossyScale.z * 0.2f);

    if (creature.GetSetPhysics.OverlapBox(groundCheck.position, boxSize,
        groundHitCol, transform.rotation,
        Managers.LayerManager.Ground | Managers.LayerManager.Wall) > 0)
    {
        groundHitCol.Initialize();
        return true;
    }

    return false;
}
```

```
// 경사 확인 // 캐릭터 밑에 Ray를 써서 땅인지 확인하는 함수
참조 1개
public bool IsSlopeCheck()
{
    if(creature.GetSetPhysics.Raycast(transform.position, Vector3.down,
        out slopeHit, SLOP_RAY_DISTANCE, Managers.LayerManager.Ground))
    {
        var angle = Vector3.Angle(Vector3.up, slopeHit.normal);
        return angle != 0f && angle < maxSlopeAngle;
    }
    return false;
}
```

- 물리 기반이라 경사가 있는 곳에 플레이어가 있으면 중력 때문에 아래로 미끄러지듯이 내려 오기 문제를 해결하기 위해 구현
- 현재 플레이어의 위치에서 아래로 Ray를 발사해 충돌체의 Normal 벡터와 윗 방향 벡터를 내적해 각도를 구한다.
- 경사진 곳에 위치하거나 땅에 발이 닿아 있을 때 중력을 OFF하는 식으로 구현

소스코드 : [PlayerController.cs](#)



빨간 점 : 현재 위치

빨간 선 : Ray를 `Vector3.Down`으로 발사한 방향

초록 선 : 피격 위치의 Normal 벡터

파란선 : `Vector3.up`

파란선과 초록선의 내적 + `ACos`를 한 각도가 $0 \sim \text{maxSlopeAngle}$
 사이에 있는 경우 경사 판정 그리고 지면에 있으면 중력을 off

9. 매달리기 & 벽 점프

```
// 콜라이더 높이 인터페이스가 있으면 대충 손 높이 썸 y축 배치 콜라이더 반지름 만큼 떨어트림
Vector3 destPos = hit.point;
if (creatureGS is IColliderInfo)
{
    IColliderInfo colliderInfo = creatureGS as IColliderInfo;

    destPos += hit.normal * colliderInfo.GetRadius();
    // 만약 Ray캐스트가 성공적으로 되면 Ray위치로 플레이어 이동
    // 플레이어 콜라이더크기를 뺀 위치에 플레이어를 이동 (콜라이더를 벽에 맞게 부착시키는 연산)
    destPos.Set(hit.point.x,
        hit.point.y - colliderInfo.GetHeight() * 0.75f, // 좌표기준이 맨밑에 있기에
        hit.point.z);
}

// 만약 Ray캐스트가 성공적으로 되면 Ray위치로 플레이어 이동
creatureGS.GetRigidBody.MovePosition(destPos);
// 벽쪽으로 보도록 이동시킨다.
creatureGS.GetRigidBody.MoveRotation(Quaternion.LookRotation(-hit.normal));
```

- 역동적인 동작을 구현하기 위해
- 플레이어가 지면에 있지 않을때 점프키를 누르면 벽에 Ray를 쏘아서 벽에 붙을 수 있으면 캐릭터를 벽에 부착 동시에 중력 OFF
- 벽에 부착된 상태로 이동키 + 점프키를 누르면 중력을 On하면서 이동 방향으로 튀어 오르게 설계

소스 코드 : [OnWallState.cs](#)

10. 씬끼리 유저 통신

```
[ClientRpc]
참조 2개
public void ChatRPC(string message)...

[ClientRpc]
참조 0개
public void RPC_Play2D(string path, Define.Sound2D type, float pitch)...

[ClientRpc]
참조 2개
public void RPC_Play3D(string path, Vector3 pos, Define.Sound3D type,
    float volume, float pitch, float minDistance, float maxDistance)...

[ClientRpc]
참조 0개
public void RPC_Play3D(string path, GameObject gameObject, bool isAttach, Define.Sound3D type,
    float volume, float pitch, float minDistance, float maxDistance)...

[ClientRpc]
참조 1개
public void RPC_Effect(string key, Vector3 spawnPos, Quaternion qut)...
```

소스 코드 : [BaseSceneNetwork.cs](#)

- 씬마다 존재하는 BaseSceneNetwork 구현
- 해당 기능은 같은 씬에 있는 플레이어 끼리 통신을 하기 위해 구현
- 해당 클래스에 채팅, 이펙트, 사운드를 동기화하는 RPC함수 구현
- 해당 씬의 유저와 RPC 통신이 필요하면 씬 고유의 BaseSceneNetwork 클래스의 RPC함수를 통해 RPC하면 씬 유저끼리 상호작용이 가능하다.

```
protected Dictionary<uint, GameObject> networkPlayerObjects =
    new Dictionary<uint, GameObject>();

protected Dictionary<uint, GameObject> networkMonsterObjects =
    new Dictionary<uint, GameObject>();

protected Dictionary<uint, GameObject> networkInAnimateObjects =
    new Dictionary<uint, GameObject>();
```

11. 인스턴스 씬

```
[ServerCallback]
참조 1개
public virtual void S_SceneObjectRemove()
{
    if (networkPlayerObjects.Count == 0)
    {
        // 씬 디렉터리에서 제거
        Managers.Scene.RemoveBaseScene(gameObject.scene);

        // 몹 스폰너 기능 중지
        foreach (MonsterSpawner monsterSpawner in monsterSpawners)
        {
            monsterSpawner.StopAllCoroutines();
        }

        // 몹 수거
        foreach (GameObject monster in networkMonsterObjects.Values)
        {
            NetworkServer.UnSpawn(monster);
        }

        // 기타 오브젝트 수거
        foreach (GameObject item in networkInAnimateObjects.Values)
        {
            NetworkServer.UnSpawn(item);
        }

        Invoke("DelayUnloadScene", 3.0f);
    }
}
```

- 유저가 던전에 입장할 때 고유한 던전이 생겨 다른 유저의 간섭을 받지 않기 위해 인스턴스 씬 구현
- 던전에 입장시 서버가 해당 씬을 동적으로 생성한 후 플레이어를 해당 씬에 배치하는 방식
- 던전 생명 주기
 - BaseScene이 현재 씬에 있는 모든 오브젝트를 관리하는 Dictionary (key : netId, value : GameObject) 존재
 - 오브젝트 생성시 BaseScene을 찾아 넣어주도록 설계
 - 인스턴스 씬에서 유저가 이탈하면 S_SceneObject_Remove 함수를 호출
 - 만약 플레이어가 0이면 실행중인 코루틴을 정지시키고 3초뒤 씬을 제거하도록 설계

소스 코드 : [BaseInstanceScene.cs](#)

12. 유튜브, 깃허브 링크



- Git HUB : <https://github.com/k660323/BossHunter>
- YouTube : <https://www.youtube.com/watch?v=ubSgPd6OHsY>